

Help

All

1

One day, the wise elders of the kingdom posed a challenge. They wanted to find special parts of the array where, if you did a special math operation called XOR on the numbers, the result would have an even number of divisors.

For example, numbers 2, 3, 5 or 6 have an even number of divisors, while 1, 4 have an odd number of divisors.

Brave adventurers stepped up to solve the puzzle. They searched through the array, looking for sections where the XOR of numbers had this special property. With each discovery, they counted the number of subarrays with XOR having an even number of divisors. Print the number of such subarrays, whose XOR has an even number of divisors.

Input Format:

- The first line of input contains a single integer n ($2 \leq n \leq 2 \times 10^5$) $\rightarrow$ the length of the array a .
- The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a[i] \leq n$).

Constraints:

- $2 \leq n \leq 2 \times 10^5$
- $1 \leq a[i] \leq n$

```
7 // class name should be "Source",
8 // otherwise solution won't be accepted
9 public class Source {
10     public static void main(String[] args) {
11
12         Scanner in = new Scanner(System.in);
13
14         // Declare the variable
15         int a;
16
17         // Read the variable from STDIN
18         a = in.nextInt();
19
20         // Output the variable to STDOUT
21         System.out.println(a);
22     }
23 }
```

Help

All

1

One day, the wise elders of the kingdom posed a challenge. They wanted to find special parts of the array where, if you did a special math operation called XOR on the numbers, the result would have an even number of divisors.

For example, numbers 2, 3, 5 or 6 have an even number of divisors, while 1, 4 have an odd number of divisors.

Brave adventurers stepped up to solve the puzzle. They searched through the array, looking for sections where the XOR of numbers had this special property. With each discovery, they counted the number of subarrays with XOR having an even number of divisors. Print the number of such subarrays, whose XOR has an even number of divisors.

Input Format:

- The first line of input contains a single integer n ($2 \leq n \leq 2 \times 10^5$) $\rightarrow$ the length of the array a .
- The second line contains n integers $a_1, a_2, \dots, a_n$ ($1 \leq a[i] \leq n$).

Constraints:

- $2 \leq n \leq 2 \times 10^5$
- $1 \leq a[i] \leq n$

```
7 // class name should be "Source",
8 // otherwise solution won't be accepted
9 public class Source {
10     public static void main(String[] args) {
11
12         Scanner in = new Scanner(System.in);
13
14         // Declare the variable
15         int a;
16
17         // Read the variable from STDIN
18         a = in.nextInt();
19
20         // Output the variable to STDOUT
21         System.out.println(a);
22     }
23 }
```

Sample Input 1:

3
3 1 2

Sample Output:

4

Explanation:

There are 4 subarrays whose XOR has an even number of divisors: [3], [3,1], [1,2], [2].

Sample Input 2:

5
4 2 1 5 3

Sample Output:

```
6
7 // Class name should be "Source",
8 // otherwise solution won't be accepted
9 public class Source {
10     public static void main(String[] args) {
11
12         Scanner in = new Scanner(System.in);
13
14         // Declare the variable
15         int a;
16
17         // Read the variable from STDIN
18         a = in.nextInt();
19
20         // Output the variable to STDOUT
21         System.out.println(a);
22     }
23 }
```

Help

All

Explanation:

There are 4 subarrays whose XOR has an even number of divisors: [3], [3,1], [1,2], [2].

Sample Input 2:

```
5
4 2 1 5 3
```

Sample Output:

```
11
```

Explanation:

There are 11 subarrays whose XOR has an even number of divisors: [4,2], [4,2,1]

[4,2,1,5], [2], [2,1], [2,1,5], [2,1,5,3], [1,5,3], [5], [5,3], [3].

Execution time limit

10 seconds

Having an issue with this question? [Report](#)

```
7 // Class name should be "Source",
8 // otherwise solution won't be accepted
9 public class Source {
10     public static void main(String[] args) {
11         Scanner in = new Scanner(System.in);
12         // Declare the variable
13         int a;
14         // Read the variable from STDIN
15         a = in.nextInt();
16         // Output the variable to STDOUT
17         System.out.println(a);
18     }
19 }
```

Test Results

Custom Input

RUN C

2. The lost puzzle of Algoria

Coding

UNSOLVED

Problem Statement

Once upon a time in the bustling city of Algoria, there lived a young programmer named Aria. Aria loved solving puzzles and challenges, especially those that involved numbers and arrays. One day, Aria stumbled upon a mysterious scroll in the attic of her grandmother's house. The scroll contained an intriguing challenge from the ancient Codemasters.

The challenge read:

"In the land of Algoria, you will be given an array of non negative integers. Your task is to rearrange this array to make it lexicographically as small as possible. However, there's a catch! You can only swap two elements in the array if the bitwise XOR of those two elements is less than 4."

Lexicographically Smallest Array: An array is considered lexicographically smaller than another array if, at the first position where the arrays differ,

two elements is less than 4."

● **Lexicographically Smallest Array:** An array is considered lexicographically smaller if at the first position where the arrays differ, the element in the first array is smaller than the element in the second array.

Input Format:

- The first line contains a single integer n ($1 \leq n \leq 2 \cdot 10^5$) — the length of the array.
- The second line contains n integers a_i ($0 \leq a_i \leq 10^9$) — the elements of the array.
- It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output Format:

- output n integers — the lexicographically smallest array that

Output format:

- output n integers — the lexicographically smallest array that can be made with any number of swaps.

Help

Constraints

- $1 \leq n \leq 2 \cdot 10^5$
- $0 \leq a_i \leq 10^9$

1

2

Evaluation Parameters:

3

Sample Input 1:

4

```
4
1 0 3 2
```

Sample Output:

```
0 1 2 3
```

1. As XOR of 0 and 1 is 1, we can swap 0 and 1 -> [0, 1, 3, 2]

2. As XOR of 3 and 2 is 1, we can swap 3 and 2 -> [0, 1, 2, 3].

All

1

Sample Input 2:

2

5

2 7 1 5 6

3

4

Sample Output:

1 5 2 6 7

Explanation:

1. As XOR of 1 and 2 is 3, we can swap 1 and 2 -> [1, 7, 2, 5, 6]

2. As XOR of 7 and 5 is 2, we can swap 7 and 5 -> [1, 5, 2, 7, 6]

3. As XOR of 7 and 6 is 1, we can swap 7 and 6 -> [1, 5, 2, 6, 7]

Execution time limit

10 seconds

Having an issue with this question? [Report](#)

11

12

13

14

15

16

17

18

19

20

21

22

23

```
Scanner in = new Scanner(System.in);  
  
// Declare the variable  
int a;  
  
// Read the variable from STDIN  
a = in.nextInt();  
  
// Output the variable to STDOUT  
System.out.println(a);  
}
```

Test Results

Custom Input